SIVF: Streaming Inverted File Indexing on GPUs

Dongfang Zhao
dzhao@uw.edu
HPDIC Lab
University of Washington, USA

Abstract

1 Introduction

1.1 Observation: Deletion is Expensive

Real-world vector search applications, such as real-time recommendation and fraud detection, increasingly operate on streaming data where timeliness is critical. These systems require a sliding window model where expired vectors must be evicted as new vectors arrive to maintain bounded memory usage. However, existing GPU-accelerated approximate nearest neighbors (ANN) indices are predominantly optimized for write-once-read-many workloads.

To quantify the gap between insertion and eviction performance, we conducted benchmarks using both the Python (precompiled package through PyPI) and C++ (local compilation from source) interfaces of Faiss [2] on an NVIDIA RTX 6000 GPU with the SIFT1M dataset [3] on the Chameleon testbed [4]. As illustrated in Figure 1, we observe a severe performance asymmetry across both implementations. In the native C++ benchmark, while inserting a batch of 10,000 vectors takes only 28.6 ms due to efficient GPU parallelism, evicting the same number of vectors incurs a latency of 202.2 ms. This constitutes a ~7.1 times slowdown, confirming that the bottleneck is fundamental to the index design rather than language binding overhead.

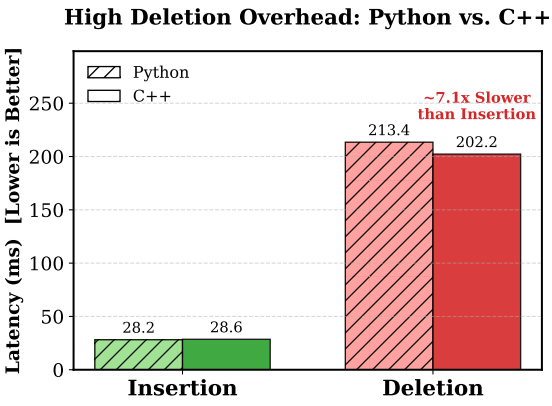

Figure 1: The High Cost of Deletion compared across Python and C++ interfaces. Even using the native C++ implementation, while GPU parallelism accelerates vector insertion (~28.6 ms), the lack of in-place GPU eviction support causes deletion latency to surge to ~202.2 ms. This represents a ~7.1x slowdown, confirming this asymmetry as the primary bottleneck in streaming scenarios regardless of the language binding.

1.2 Root Cause: The Missing Operator

Our analysis of the Faiss source code [1] reveals that the performance asymmetry shown in Figure 1 is not merely an optimization issue, but a fundamental feature gap enforced by the library's class hierarchy.

In the Faiss codebase, the data eviction interface is defined in the abstract base class `faiss::Index` via the `remove_ids` virtual method. By default, this base implementation throws an exception, indicating a lack of support. While CPU-based implementations such as `IndexIVF` and `IndexFlat` explicitly override this method to provide efficient, in-memory deletion logic (e.g., via `std::vector` manipulation or `memmove`), their GPU counterparts fail to do so. Specifically, the GPU base class `faiss::gpu::GpuIndex` and its derivatives (e.g., `GpuIndexIVF`) inherit the exception-throwing base implementation without providing a GPU-native override.

This implementation gap necessitates a fallback to a distinct *CPU-GPU Roundtrip* pattern for eviction, as observed in our benchmarks. Consequently, to delete even a single vector from a GPU-resident index, the system must transfer the entire index state to host memory, perform the deletion using CPU primitives, and re-upload the modified index to the device.

1.3 Technical Challenges

The absence of a GPU-native `remove_ids` stems from the complexity of managing dynamic data structures in VRAM. Unlike the CPU's `IndexIVFFlat`, which utilizes dynamic arrays (STL vectors) to manage inverted lists, GPU indices rely on pre-allocated, contiguous memory blocks to maximize memory bandwidth and parallelism. Implementing in-place deletion on the GPU would require:

- **Dynamic Memory Management:** Frequent reallocation and compaction of inverted lists within VRAM to prevent fragmentation.
- **Complex Synchronization:** Fine-grained locking mechanisms to prevent race conditions when thousands of concurrent GPU threads attempt to modify shared list structures.
- **Reverse Mapping Overhead:** Standard IVF indices do not maintain an ID-to-Location mapping. Locating a specific ID for deletion requires scanning all inverted lists, an operation with $O(N)$ complexity that negates the benefits of GPU acceleration.

These architectural constraints render in-place GPU eviction prohibitively complex to implement efficiently, forcing the current IO-bound workaround.

1.4 Proposed Solution: The SIVF Architecture

To overcome the architectural hurdles identified in Section 1.3, we propose SIVF (Streaming Inverted File), a new GPU indexing

architecture designed specifically for high-throughput insertion and deletion, and eventually for efficient streaming data analytics on GPUs. SIVF introduces three key mechanisms to resolve the identified bottlenecks:

Slab-based Memory Management. To mitigate the overhead of **Dynamic Memory Management**, SIVF abandons the contiguous memory layout of standard IVF. Instead, we propose a *Slab Allocation System*. The GPU memory is pre-allocated as a pool of fixed-size blocks (e.g., 4KB pages). Inverted lists are implemented as chains of these blocks. This design obviates the need for costly memory reallocation and data compaction. When a list grows, a new block is simply linked from the free pool using atomic operations. This ensures that memory fragmentation is virtually eliminated and ingestion latency remains constant regardless of list size.

Bitmap-based Lazy Eviction. To address **Complex Synchronization**, we decouple logical deletion from physical data removal. We introduce a *Validity Bitmap* associated with each memory block. Eviction is implemented as a *Lazy Deletion* strategy: removing a vector involves only a single atomic bit-flip in the bitmap to mark the slot as invalid. This lock-free approach allows thousands of concurrent threads to perform deletions without contention. Physical reclamation of memory blocks is deferred and performed asynchronously by a background kernel only when the utilization of a block drops below a threshold, effectively hiding the cleanup cost.

GPU-Resident Address Table. To eliminate the **Reverse Mapping Overhead**, SIVF maintains a compact, GPU-resident *Address Translation Table* (ATT). This table maps each active Vector ID to its precise physical location (composed of `BlockID` and `SlotOffset`). By compressing this location metadata into a single 64-bit integer, we minimize the memory footprint. This structure allows the eviction kernel to locate any target vector in $O(1)$ time, avoiding the prohibitive $O(N)$ scan required by current libraries [1].

By synthesizing these three techniques, SIVF enables true in-place modification of the index directly on the GPU, breaking the dependency on the host CPU and eliminating the roundtrip bottleneck. As a result, this paves the road for high-throughput, low-latency streaming vector search applications on GPU platforms.

1.5 Contributions

In summary, this paper makes the following contributions:

- We identify a critical performance bottleneck in existing GPU-accelerated ANN indices stemming from the lack of native support for in-place data eviction. We analyze the root causes of this limitation and quantify its impact through empirical benchmarks.
- We propose SIVF, a novel indexing architecture that enables efficient streaming insertion and deletion directly on the GPU. SIVF introduces slab-based memory management, bitmap-based lazy eviction, and a GPU-resident address translation table to overcome the challenges of dynamic data structures in VRAM.
- We implement a prototype of SIVF and evaluate its performance against state-of-the-art GPU ANN libraries. Our results demonstrate that SIVF significantly reduces eviction latency, paving the way for real-time streaming vector search applications on GPUs.

2 Related Work

3 Design and Implementation

[Dongfang: TODO]

4 Evaluation

Our implementation of SIVF is built upon the Faiss library [1], extending its GPU index classes to incorporate our proposed mechanisms. We conduct a comprehensive evaluation of SIVF against the native Faiss GPU index implementations, focusing on insertion and eviction performance under streaming workloads. All of our implementation and evaluation code are open-sourced as a side project to *ElasticIVF* hosted on GitHub[1].

5 Conclusion

In this paper, we identified a critical performance bottleneck in existing GPU-accelerated ANN indices stemming from the lack of native support for in-place data eviction. We analyzed the root causes of this limitation and proposed SIVF, a new indexing architecture that enables efficient streaming insertion and deletion directly on the GPU. By introducing slab-based memory management, bitmap-based lazy eviction, and a GPU-resident address translation table, SIVF overcomes the challenges of dynamic data structures in VRAM. Our evaluation demonstrates that SIVF significantly reduces eviction latency, paving the way for real-time streaming vector search applications on GPUs.

Acknowledgment

Results presented in this paper were obtained using the Chameleon testbed supported by the National Science Foundation.

References

[1] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. The faiss library.

[2] Johnson, J., Douze, M., and Jégou, H. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data 7*, 3 (2021), 535–547.

[3] Jégou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence 33*, 1 (2011), 117–128.

[4] Keahey, K., Anderson, J., Zhen, Z., Riteau, P., Ruth, P., Stanzione, D., Cevik, M., Colleran, J., Gunawi, H. S., Hammock, C., Mambretti, J., Barnes, A., Halbach, F., Rocha, A., and Stubbs, J. Lessons learned from the chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.

[1] https://github.com/hpdic/ElasticIVF